Experiment 7: Expression Conversion using Stack

Problem Statement:
Implement stack as an ADT and apply it for different expression conversions (infix to postfix or prefix, prefix to postfix or prefix to infix, postfix to infix or prefix)

Objectives:
i. To study stack and its operations.
ii. To study the importance of expression conversions.

Theory:
Stack: Special case of ordered list also called as restricted/contr-olled list where insertion and deletion happen at only one end called as top of stack (homogeneous collection of elements.) Elements are added to and removed from the top of the stack (the most recently added items are at the top of the stack). The last element to be added is the first to be removed (LIFO). Only access to the stack is the top element.

Basic Stack Operations:
- isEmpty() - Checking stack is empty.
- isFull() - Checking stack is full.
- push() - Pushing elements on top of stack
- pop() - Popping top element from stack.

Applications:
i. Expression evaluation
ii. Infix to Postfix and Prefix conversion
iii. Function call management
iv. Recursion
v. Parentheses matching

vi. Undo/Redo operations in editors
vii. Browser back and forward navigation
viii. Depth-first search (DFS) in graphs.
ix. Syntax parsing in compilers
x. String reversal

Expression conversion is the process of converting an arithmetic expression from one notation to another using stack. The three common notat^n are:
i. Infix: Operator is between operands (eg A+B)
ii. Prefix: Operator is before operands (eg +AB)
iii. Postfix: Operator is after operands (eg AB+)
A stack is used to temporarily store operators and parentheses during the conversion.

Implementation:

- Platform

i. 64-bit Open source linux or its derivatives.
ii. Open Source C Programming tool like gcc/Eclipse Editor

- Test conditions.

Test case	Input Type	Input Expression	Target Conv.	Expected Output
TC01	Infix	(A+B)* C-D	Postfix	AB+C*D-
TC02	Infix	A+(B*C/D)	Prefix	+A*/BCD
TC03	Postfix	AB+C*D-	Infix	((A+B)*C)-D
TC04	Postfix	AB+C*D-	Prefix	-*+ABCD
TC05	Prefix	-*+ABCD	Infix	((A+B)*C)-D
TC06	Prefix	-*+ABCD	Postfix	AB+C*D-

- Pseudo Code:

```
Algorithm push(x):
        if top == MAX-1:
            display "Stack Overflow"
            return
        set top = top + 1
        set stack[top] = x
end push

Algorithm pop():
        if top == -1
            display "Stack Underflow"
            return NULL
        set item = stack[top]
        set top = top - 1
        return item
end pop

Algorithm precedence(symbol):
        if symbol == '^' : return 3
        else if symbol == '*' or symbol == '/' : return 2
        else if symbol == '+' or symbol == '-' : return 1
        else : return 0
end precedence
```

```
Algorithm InfixtoPostfix(inexp):
    set k=0, i=0
    while inexp[i]!='\0':
        set token = inexp[i]
        if token is an operand:
            postfix[k] = token
            k = k+1
        else if token == '(':
            push('(')
        else if token == ')':
            while top != -1 and stack[top] != '(':
                postfix[k] = pop()
                k = k+1
            pop()
        else (token is an operator):
            while top!=-1 and precedence(stack[top] >=
            precedence(token):
                postfix[k] = pop()
                k = k+1
            push(token)
        set i = i+1
    while top != -1:
        postfix[k] = pop()
        k = k+1
    set postfix[k] = '\0'
    return postfix
End InfixtoPostfix
```

```
Algorithm Postfix to Infix (postexp):
        set len = length of postexp
        for i=0 to len-1:
            set token = postexp[i]
                if token is an operand:
                    push (string (token))
                else (token is an operator):
                    op2 = pop()
                    op1 = pop()
                    expr = strcat ('(', op1, token, op2, ')')
                    push (expr)
            return pop()
end Postfix to Infix
```

```
Algorithm PostfixtoPrefix (postexp):
        set len = length of postexp
        for i=0 to len-1:
            set token = postexp[i]
                if token is an operand:
                    push (string (token))
                else (token is an operator):
                    op2 = pop()
                    op1 = pop()
                    expr = strcat (token, op1, op2)
                    push (expr)
            return pop()
end PostfixtoPrefix
```

```
Algorithm PrefixtoInfix(preexp):
        set len = length of preexp
        for i = len-1 down to 0:
            set token = preexp[i]
                if token is an operand:
                    push(string(token))
                else (token is an operator):
                    op1 = pop()
                    op2 = pop()
                    expr = strcat('(', op1, token, op2, ')')
                    push(expr)
            return pop()
end PrefixtoInfix
```

```
Algorithm PrefixtoPostfix(preexp):
        set len = length of preexp
        for i = len-1 down to 0:
            set token = preexp[i]
                if token is an Operand:
                    push(string(token))
                else (token in an operator):
                    op1 = pop()
                    op2 = pop()
                    expr = strcat(op1, op2, token)
                    push(expr)
            return pop()
end PrefixtoPostfix
```

Algorithm main():

Display menu:

1. Infix to Postfix
2. Postfix to Infix
3. Postfix to Prefix
4. Prefix to Infix
5. Prefix to Postfix
6. Exit

Read choice

switch (choice):

case 1:

Read infix expression

call InfixtoPostfix()

display result

break

case 2:

Read postfix expression

call PostfixtoInfix()

display result

break

case 3:

Read postfix expression, call PostfixtoPrefix(), display result, break.

case 4:

Read prefix expression, call PrefixtoInfix(), display result; break

case 5:

Read prefix expression, call PrefixtoPostfix(), display result, break

case 6:

Terminate program

Default:

Display "Invalid Choice"

Repeat until user selects exit

End main

- Time & Space complexity:

Oper" /Func	Time Comp.	Space Comp
push()	O(1)	O(1)
pop()	O(1)	O(1)
Infix to Postfix	O(N)	O(N)
Postfix to Infix	O(N)	O(N)
Postfix to Prefix	O(N)	O(N)
Prefix to Infix	O(N)	O(N)
Prefix to Postfix	O(N)	O(N)

Conclusion:

Thus, implemented Stack Operations assignment using Array concepts. This system is able to apply different expression conversion.

FAQs:

1. What is the advantage of prefix & postfix over infix expression?

Ans. The primary advantage of prefix & postfix expressions over the infix expressions is that they eliminate the need for parentheses and operator precedence rules, enabling computers to evaluate expression unambiguously in a single pass.

2. Why is stack in expression conversion?

Ans. A stack is used in expression conversion because its LIFO behaviour allows temporary holding of operators, operands, or parentheses until they can be emitted in the correct order dictated by operators precedence.

3. Give Stack full and Stack empty conditions for stack.

Ans For an array-based stack of size MAX using index variable top the stack full (overflow) condition is top == MAX - 1 and the stack empty (underflow) condition is top == -1.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

// Stack for Operators / Characters
typedef struct {
    char items[MAX];
    int top;
} CharStack;

void initCharStack(CharStack *s) { s->top = -1; }
int isCharEmpty(CharStack *s) { return s->top == -1; }
int isCharFull(CharStack *s) { return s->top == MAX - 1; }
void pushChar(CharStack *s, char c) {
    if (!isCharFull(s)) s->items[++(s->top)] = c;
}
char popChar(CharStack *s) {
    return (!isCharEmpty(s)) ? s->items[(s->top)--] : '\0';
}
char peekChar(CharStack *s) {
    return (!isCharEmpty(s)) ? s->items[s->top] : '\0';
}

// Stack for Expressions / Strings
typedef struct {
    char items[MAX][MAX];
    int top;
} StringStack;

void initStringStack(StringStack *s) { s->top = -1; }
int isStringEmpty(StringStack *s) { return s->top == -1; }
```

```
void pushString(StringStack *s, const char *str) {
    if (s->top < MAX - 1) strcpy(s->items[++(s->top)], str);
}
void popString(StringStack *s, char *output) {
    if (!isStringEmpty(s)) {
        strcpy(output, s->items[(s->top)--]);
    } else {
        output[0] = '\0';
    }
}

// Operator Precedence Helper
int precedence(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

// 1. Infix to Postfix
void infixToPostfix(char *infix, char *postfix) {
    CharStack s;
    initCharStack(&s);
    int k = 0;
    for (int i = 0; infix[i] != '\0'; i++) {
        char token = infix[i];
        if (isalnum(token)) {
            postfix[k++] = token;
        } else if (token == '(') {
            pushChar(&s, token);
        } else if (token == ')') {
            while (!isCharEmpty(&s) && peekChar(&s) != '(') {
                postfix[k++] = popChar(&s);
            }
            popChar(&s); // Discard '('
        } else {
```

```
            while (!isCharEmpty(&s) && precedence(peekChar(&s)) >=
precedence(token)) {
                postfix[k++] = popChar(&s);
            }
            pushChar(&s, token);
        }
    }
    while (!isCharEmpty(&s)) {
        postfix[k++] = popChar(&s);
    }
    postfix[k] = '\0';
}

// 2. Postfix to Infix
void postfixToInfix(char *postfix, char *infix) {
    StringStack s;
    initStringStack(&s);
    for (int i = 0; postfix[i] != '\0'; i++) {
        char token = postfix[i];
        if (isalnum(token)) {
            char str[2] = {token, '\0'};
            pushString(&s, str);
        } else {
            char op2[MAX], op1[MAX], expr[MAX];
            popString(&s, op2);
            popString(&s, op1);
            snprintf(expr, sizeof(expr), "(%s%c%s)", op1, token, op2);
            pushString(&s, expr);
        }
    }
    popString(&s, infix);
}

// 3. Postfix to Prefix
void postfixToPrefix(char *postfix, char *prefix) {
    StringStack s;
```

```
    initStringStack(&s);
    for (int i = 0; postfix[i] != '\0'; i++) {
        char token = postfix[i];
        if (isalnum(token)) {
            char str[2] = {token, '\0'};
            pushString(&s, str);
        } else {
            char op2[MAX], op1[MAX], expr[MAX];
            popString(&s, op2);
            popString(&s, op1);
            snprintf(expr, sizeof(expr), "%c%s%s", token, op1, op2);
            pushString(&s, expr);
        }
    }
    popString(&s, prefix);
}

// 4. Prefix to Infix
void prefixToInfix(char *prefix, char *infix) {
    StringStack s;
    initStringStack(&s);
    int len = strlen(prefix);
    for (int i = len - 1; i >= 0; i--) {
        char token = prefix[i];
        if (isalnum(token)) {
            char str[2] = {token, '\0'};
            pushString(&s, str);
        } else {
            char op1[MAX], op2[MAX], expr[MAX];
            popString(&s, op1);
            popString(&s, op2);
            snprintf(expr, sizeof(expr), "(%s%c%s)", op1, token, op2);
            pushString(&s, expr);
        }
    }
    popString(&s, infix);
```

```
}

// 5. Prefix to Postfix
void prefixToPostfix(char *prefix, char *postfix) {
    StringStack s;
    initStringStack(&s);
    int len = strlen(prefix);
    for (int i = len - 1; i >= 0; i--) {
        char token = prefix[i];
        if (isalnum(token)) {
            char str[2] = {token, '\0'};
            pushString(&s, str);
        } else {
            char op1[MAX], op2[MAX], expr[MAX];
            popString(&s, op1);
            popString(&s, op2);
            snprintf(expr, sizeof(expr), "%s%s%c", op1, op2, token);
            pushString(&s, expr);
        }
    }
    popString(&s, postfix);
}

int main() {
    int choice;
    char input[MAX], output[MAX];
    while (1) {
        printf("\n=== STACK EXPRESSION CONVERSION MENU ===\n");
        printf("1. Infix to Postfix\n");
        printf("2. Postfix to Infix\n");
        printf("3. Postfix to Prefix\n");
        printf("4. Prefix to Infix\n");
        printf("5. Prefix to Postfix\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) break;
```

```
        if (choice == 6) {
            printf("Exiting program. Goodbye!\n");
            break;
        }

        printf("Enter expression: ");
        scanf("%s", input);

        switch (choice) {
            case 1:
                infixToPostfix(input, output);
                printf("Postfix Expression: %s\n", output);
                break;
            case 2:
                postfixToInfix(input, output);
                printf("Infix Expression: %s\n", output);
                break;
            case 3:
                postfixToPrefix(input, output);
                printf("Prefix Expression: %s\n", output);
                break;
            case 4:
                prefixToInfix(input, output);
                printf("Infix Expression: %s\n", output);
                break;
            case 5:
                prefixToPostfix(input, output);
                printf("Postfix Expression: %s\n", output);
                break;
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

Input:

Switch Option	Resp. Expression
1	(A+B)*C-D
2	AB+C*D-
3	AB+C*D-
4	-*+ABCD
5	-*+ABCD
6	Exit

Output:

```
C:\Users\shrey\CLionProjects\untitled\Ass1.exe

=== STACK EXPRESSION CONVERSION MENU ===
1. Infix to Postfix
2. Postfix to Infix
3. Postfix to Prefix
4. Prefix to Infix
5. Prefix to Postfix
6. Exit
Enter your choice:1

Enter expression:(A+B)*C-D

Postfix Expression: AB+C*D-

=== STACK EXPRESSION CONVERSION MENU ===
1. Infix to Postfix
2. Postfix to Infix
3. Postfix to Prefix
4. Prefix to Infix
5. Prefix to Postfix
6. Exit
Enter your choice:2

Enter expression:AB+C*D-

Infix Expression: (((A+B)*C)-D)
```

```
=== STACK EXPRESSION CONVERSION MENU ===
1. Infix to Postfix
2. Postfix to Infix
3. Postfix to Prefix
4. Prefix to Infix
5. Prefix to Postfix
6. Exit
Enter your choice:3

Enter expression:AB+C*D-

Prefix Expression: -*+ABCD

=== STACK EXPRESSION CONVERSION MENU ===
1. Infix to Postfix
2. Postfix to Infix
3. Postfix to Prefix
4. Prefix to Infix
5. Prefix to Postfix
6. Exit
Enter your choice:4

Enter expression:-*+ABCD

Infix Expression: (((A+B)*C)-D)
```

```
=== STACK EXPRESSION CONVERSION MENU ===
1. Infix to Postfix
2. Postfix to Infix
3. Postfix to Prefix
4. Prefix to Infix
5. Prefix to Postfix
6. Exit
Enter your choice:3

Enter expression:AB+C*D-

Prefix Expression: -*+ABCD

=== STACK EXPRESSION CONVERSION MENU ===
1. Infix to Postfix
2. Postfix to Infix
3. Postfix to Prefix
4. Prefix to Infix
5. Prefix to Postfix
6. Exit
Enter your choice:4

Enter expression:-*+ABCD

Infix Expression: (((A+B)*C)-D)
```